VIT®
CHENNAI
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

REVIEW REPORT

RESUMESCAN: Your Personalized Resume Analysis

Name: T Vishal

Reg No.: 24BAI1525

Faculty name: Dr. Prem Sankar

Course name: Artificial Intelligence (BCSE306L)

Slot: F1

Core Concept: AI-Based Resume Analysis and Recommendation System

Abstract

In recent years, the recruitment process has rapidly transitioned from traditional hiring practices to digital recruitment platforms. Most organizations now publish job opportunities online and receive applications electronically, leading to a significant increase in the number of resumes submitted for each position. Companies commonly utilize automated screening tools such as Applicant Tracking Systems (ATS) to manage this volume efficiently. However, many job applicants—especially students and fresh graduates—lack awareness of how these systems evaluate resumes. As a result, even qualified candidates may fail to progress in the recruitment process due to improper formatting, missing keywords, or poorly structured content.

This project proposes the design and development of a **Personalized Artificial Intelligence-Based Resume Analyzer**, a web-based application intended to assist candidates in improving their resumes before submitting job applications. Unlike conventional automated screening systems that are designed for recruiters, the proposed system focuses on empowering applicants by providing self-evaluation and guided improvement. Users upload resumes in PDF format, which are then automatically processed by the system. The document is parsed to extract textual content, and Natural Language Processing (NLP) techniques are applied to evaluate the resume.

The system analyzes multiple resume quality parameters including presence of essential sections (education, skills, projects, and experience), keyword relevance, clarity of writing, formatting structure, readability, and ATS compatibility. Based on these factors, the analyzer generates a quantitative score along with detailed feedback and improvement suggestions. The purpose is not to reject resumes but to guide candidates in creating optimized and targeted resumes suitable for different job roles and companies.

The application is implemented using modern web technologies with JavaScript and JSX as the primary development environment, intentionally adopting a more challenging approach compared to traditional Python-based implementations. Additionally, the system incorporates a cloud storage module that allows users to maintain multiple versions of resumes tailored to different organizations. This enables applicants to personalize applications and improve their chances of passing automated screening processes and attracting recruiter attention.

The proposed solution acts as an intelligent career-assistance tool that bridges the knowledge gap between recruitment automation systems and job seekers. By providing transparent evaluation and actionable recommendations, the system enhances candidate preparedness, improves resume quality, and increases the likelihood of securing interview opportunities. The project demonstrates a practical real-world application of Artificial Intelligence and Natural Language Processing in the domain of employability enhancement and candidate-centric recruitment support.

1. Introduction

The hiring process in the modern job market has undergone a significant transformation due to the rapid adoption of digital technologies. Organizations now depend heavily on online recruitment platforms to advertise vacancies and collect applications. Instead of reviewing printed resumes manually, companies receive applications electronically and process them using automated systems. Among these systems, Applicant Tracking Systems (ATS) play a major role in filtering candidates before the recruiter reviews the applications. These systems analyze resumes based on predefined rules such as keyword matching, formatting structure, and section completeness.

Although ATS technology helps recruiters manage a large number of applications efficiently, it introduces a major challenge for job applicants. Many candidates, particularly students and entry-level job seekers, do not understand how resumes are evaluated by automated screening systems. A candidate may possess the required skills and qualifications but may still be rejected due to missing keywords, improper organization of sections, poor formatting, or lack of clarity in describing experience. As a result, the resume becomes a barrier rather than a representation of the candidate's abilities.

Most existing resume analysis or ATS-related tools are designed from the perspective of recruiters. They aim to shortlist candidates rather than assist them. This creates a gap where applicants receive rejection notifications but are not provided with feedback explaining what went wrong. Consequently, candidates repeatedly submit ineffective resumes without understanding how to improve them.

The proposed project addresses this problem by introducing a **candidate-centric personalized resume analyzer**. Instead of functioning as a filtering mechanism for companies, the system acts as a guidance tool for applicants. The primary objective is to help users evaluate and improve their resumes before applying for jobs. The application allows a user to upload a resume in PDF format and automatically analyzes its contents. The system extracts text from the document, evaluates the presence of essential sections such as education, skills, projects, and experience, and checks whether the resume meets common ATS readability and formatting standards.

Based on the analysis, the system generates a score representing the overall quality of the resume along with specific suggestions for improvement. These suggestions may include adding missing sections, improving skill representation, refining project descriptions, or modifying formatting structure. By implementing this feedback, candidates can tailor their resumes according to specific companies and job roles, thereby increasing their chances of clearing automated screening systems and receiving interview opportunities.

An important aspect of this project is the choice of development technology. Resume analysis systems are commonly implemented using Python due to its strong support for Natural Language Processing libraries. However, in this project, JavaScript and JSX were selected as the core technologies to create a fully web-based interactive application. This decision was made to explore the implementation of Artificial Intelligence concepts in a client-side web environment and to make the project more technically challenging. The system also integrates a cloud storage backend that enables users to store and manage multiple versions of resumes customized for different companies.

Thus, the proposed system serves as a personal resume coach rather than a recruitment filter. It empowers candidates with actionable insights, improves transparency in resume evaluation, and helps bridge the gap between applicants and automated recruitment technologies. By enabling users to understand how resumes are assessed, the system contributes to improving employability and job preparedness in a practical and accessible manner.

2. Literature Review

The increasing use of digital recruitment systems has encouraged researchers and developers to design automated resume screening and analysis tools. Most of the existing work in this domain focuses on assisting recruiters in filtering candidates rather than helping applicants improve their resumes. These systems typically rely on Natural Language Processing (NLP), keyword matching, and machine learning techniques to evaluate candidate suitability.

Early recruitment automation systems primarily used keyword-based filtering methods. In such systems, resumes were compared with job descriptions to identify matching skills and qualifications. If a resume lacked specific keywords present in the job posting, the system automatically rejected it even if the candidate possessed relevant knowledge expressed in different wording. This highlighted a limitation of ATS systems — they evaluated formatting and keyword presence rather than actual competence [1].

To overcome this limitation, researchers introduced Natural Language Processing techniques for resume parsing. Resume parsing involves extracting structured information such as name, education, experience, and skills from unstructured text documents. NLP-based parsers improved the ability of systems to interpret resumes by identifying contextual meaning instead of simple word matching. Techniques such as tokenization, part-of-speech tagging, and named entity recognition were applied to identify sections and important entities within resumes [2].

Machine learning approaches were later introduced to improve candidate classification. These systems trained models on previously selected candidates and attempted to predict whether a new applicant matched the desired profile. Although such systems increased recruiter efficiency, they were still primarily employer-centric and did not provide feedback to applicants. The candidate only received a rejection without understanding the reason [3].

Recent research also explored semantic similarity analysis between resumes and job descriptions. Instead of matching only exact keywords, these methods analyze conceptual similarity using vector representations of text. Word embeddings and contextual representations allow systems to understand that related terms such as “programming,” “software development,” and “coding” may represent similar competencies [4]. This improved matching accuracy but continued to function mainly as a screening tool.

Some career-assistance platforms introduced resume scoring features to evaluate formatting quality and readability. These tools provided general feedback such as checking grammar, improving structure, or adding measurable achievements. However, most of them were static rule-based systems and lacked personalization according to individual users or target companies [5].

Another important development in this field is the use of AI-based writing assistants. These systems analyze language clarity, tone, and readability to improve professional communication. Applying similar analysis to resumes can help candidates present achievements more effectively and professionally. Such approaches demonstrate the potential of AI not only for filtering applicants but also for guiding them [6].

Despite the advancements in resume parsing and automated recruitment, a significant limitation remains: existing systems are primarily designed for recruiters and organizations. There is comparatively less focus on empowering candidates to understand ATS evaluation criteria. Many job seekers do not know why their resumes fail screening and therefore cannot improve them effectively.

The proposed personalized resume analyzer differs from prior systems by shifting the focus from recruitment automation to candidate assistance. Instead of rejecting applications, the system evaluates resumes, assigns a score, and provides improvement suggestions. Additionally, the system allows users to maintain multiple resumes for different companies through a cloud-based storage approach. This makes the application not a recruitment filter but a learning and preparation tool for applicants.

By combining document parsing, resume scoring, and personalized feedback, the project aims to bridge the gap between ATS-based recruitment systems and job seekers, thereby enhancing employability and application success rates.

3. Research Gap Analysis

Although automated resume screening and recruitment technologies have advanced significantly, most existing solutions are designed primarily to support recruiters and organizations rather than job applicants. Applicant Tracking Systems (ATS) are developed to reduce recruiter workload by automatically filtering large volumes of applications. These systems focus on identifying suitable candidates by matching resume content with job descriptions. However, they do not explain the rejection criteria to applicants, leaving candidates unaware of what went wrong in their resumes [1].

One major limitation of current recruitment automation tools is the lack of transparency. ATS software evaluates resumes based on keyword presence, formatting compatibility, and structural organization, but candidates rarely receive feedback. As a result, students and fresh graduates often submit multiple applications without understanding why they are not shortlisted. Even qualified candidates may be rejected due to missing keywords, improper section headings, or non-standard formatting [2].

Existing resume evaluation platforms provide only generic advice such as “improve formatting” or “add more skills.” These tools usually rely on rule-based checks and do not provide personalized suggestions. They also do not help users adapt resumes for different companies or job roles. In real-world job applications, candidates often need to tailor their resumes according to the specific organization and role requirements. Current systems rarely support maintaining multiple resume versions for this purpose.

Another research gap is that most academic and industrial projects related to resume analysis are employer-centric. They aim to rank applicants, classify candidates, or shortlist resumes automatically. While these approaches improve recruitment efficiency, they do not assist candidates in learning how ATS systems evaluate resumes. Therefore, applicants remain at a disadvantage when interacting with automated hiring platforms [3].

Furthermore, many resume analyzers are implemented as offline tools or simple document scanners. They lack an integrated platform where users can repeatedly upload, evaluate, improve, and store their resumes. Candidates frequently create different resumes for different companies, but managing multiple versions manually is difficult and unorganized. A structured storage and improvement system is generally missing in most existing solutions.

Another important gap is technological accessibility. Several research implementations are developed in Python-based environments and remain limited to experimental or academic usage. They are not always provided as interactive web applications accessible to students and entry-level job seekers. There is a need for a user-friendly web-based system that allows easy interaction without requiring technical expertise.

The proposed project addresses these limitations by introducing a candidate-centric personalized resume analyzer. Instead of filtering applicants, the system evaluates resumes and provides a score along with specific suggestions for improvement. The goal is to educate users about ATS expectations rather than silently rejecting them. Additionally, the system allows users to store multiple resume versions for different companies using a cloud storage mechanism, enabling structured resume management.

Another distinguishing feature of the project is the choice of implementation technology. While such systems are commonly implemented using Python due to strong AI libraries, this project intentionally adopts a JavaScript-based web approach. By building the analyzer using JavaScript and JSX with a browser-based interface, the project demonstrates how intelligent applications can be developed within a modern web development environment. This makes the system accessible, interactive, and platform-independent.

Therefore, the research gap identified can be summarized as follows:

- Existing systems assist recruiters, not candidates.
- Lack of feedback prevents applicants from improving resumes.
- No support for maintaining multiple tailored resumes.
- Limited transparency in ATS evaluation criteria.

- Absence of an accessible web-based personalized improvement tool.

By addressing these gaps, the proposed system transforms resume analysis from a rejection-based mechanism into a learning-oriented guidance platform, helping candidates improve their chances of passing ATS screening and presenting themselves as strong applicants to recruiters.

4. System Overview

The proposed system is a **Personalized AI Resume Analyzer**, a web-based application developed to assist job applicants in evaluating and improving their resumes before applying to companies. Unlike traditional recruitment software that filters candidates for recruiters, this system is designed specifically for **candidates themselves**. The platform helps users understand how their resumes are interpreted by automated screening tools such as Applicant Tracking Systems (ATS) and provides guidance to improve acceptance probability.

The system allows a user to upload a resume in PDF format through a browser interface. Once uploaded, the application processes the document, extracts textual content, analyzes the information, and generates a performance score along with improvement suggestions. The system acts as a **practice evaluator**, meaning the resume is not rejected but instead reviewed and explained to the user in an understandable manner.

The primary goal of the system is to help candidates tailor resumes according to different companies and job roles. Many applicants submit the same resume to multiple organizations, but companies look for specific keywords, relevant skills, and structured presentation. The analyzer helps users identify missing components such as skills, projects, or experience descriptions, and encourages them to modify their resume accordingly. By doing so, candidates can create multiple optimized resumes and improve their chances of clearing ATS screening [2].

The application operates as a browser-based platform implemented using **JavaScript and JSX (React framework)**. Although resume analysis systems are commonly implemented in Python due to its strong Artificial Intelligence ecosystem, this project intentionally adopts a web-technology approach. The purpose is to demonstrate that intelligent document analysis and feedback generation can also be performed within a modern web development environment. This makes the system interactive, platform-independent, and easy for users to access without installing software.

The system consists of several major functional modules:

1. User Interface Module
Provides a web page where users can upload resumes and view feedback. The interface is designed to be simple and user-friendly so that even non-technical users can operate it easily.

2. Document Processing Module
Reads the uploaded PDF file and extracts textual content. This module converts the resume from a document format into analyzable text.

3. Resume Analysis Module
Evaluates the extracted text using predefined resume quality metrics such as presence of sections, keyword relevance, skill representation, and writing clarity.

4. Feedback Generation Module
Produces a numerical score and improvement suggestions. Instead of rejection, the system explains what the user should change or add.

5. Resume Storage Module (Cloud Backend)
Allows users to store different versions of resumes for different companies. This helps candidates maintain organized resume records and reuse them later.

The overall working concept is simple:

Upload → Extract Text → Analyze → Score → Suggest Improvements → Store Resume

Through this workflow, the system becomes not just a resume checker but a learning tool. Users can repeatedly upload improved resumes and observe how their score changes. This iterative improvement process helps candidates gradually develop professional resumes aligned with industry expectations [3].

In summary, the system provides a practical and user-oriented career assistance platform. Instead of automating hiring decisions, it empowers applicants by making resume evaluation understandable, transparent, and actionable. The web-based implementation further ensures accessibility and ease of use for students, fresh graduates, and job seekers.

5. Detailed Methodology

The methodology of the proposed Personalized AI Resume Analyzer describes the step-by-step procedure followed by the system to process a resume and generate meaningful feedback for the candidate. The objective is not merely to read the resume but to **interpret it similarly to an Applicant Tracking System (ATS)** and then explain the results to the user in a human-understandable manner.

The system follows a sequential processing pipeline beginning from resume upload and ending with feedback generation and storage.

Step 1: Resume Upload

The process begins when a user accesses the web application and uploads a resume file in **PDF format** through the browser interface.

The system validates:

- File type (must be PDF)
- File readability
- File size constraints

PDF is selected because it is the most commonly used format in job applications and preserves formatting across devices. Accepting a standard format ensures consistency and compatibility with automated screening systems [1].

Step 2: Document Parsing and Text Extraction

After upload, the system extracts textual content from the resume.
The uploaded PDF is not directly analyzable because it is a structured document containing fonts, layout objects, and images. Therefore, a **document parsing module** converts the resume into plain text.

The extraction process:

1. Read binary PDF file
2. Interpret document objects
3. Convert content into machine-readable text
4. Remove non-informational elements (spacing, symbols, page breaks)

The system identifies headings such as:

- Education
- Experience
- Skills
- Projects
- Certifications

This stage converts the resume into structured text data suitable for analysis using Natural Language Processing techniques [2].

Step 3: Pre-Processing of Resume Content

The extracted text undergoes preprocessing before evaluation.

Preprocessing includes:

- Lowercasing text
- Removing special characters
- Eliminating stop words (a, the, is, etc.)
- Sentence segmentation
- Tokenization (splitting text into words)

This stage improves analysis accuracy by reducing noise and standardizing the data. Preprocessing is a common step in NLP-based document analysis systems [3].

Step 4: Section Identification

The system checks whether essential resume sections are present.

The analyzer searches for keywords and headings to detect:

- Contact information
- Education
- Skills
- Projects
- Work Experience
- Certifications

If a section is missing, the system flags it as a weakness. A resume missing critical sections often gets rejected by ATS software before recruiter review [4].

Step 5: Keyword and Skill Matching

The analyzer evaluates whether the resume contains **relevant job-oriented keywords**.

The system:

- Maintains a predefined skill database
- Compares resume words with skill keywords
- Calculates keyword density

Examples:

- Programming: Java, Python, C++, JavaScript

- Technologies: React, SQL, Machine Learning, Cloud
- Soft skills: communication, teamwork, leadership

ATS systems rely heavily on keyword matching when filtering resumes [5].
If keywords are insufficient or unrelated, the candidate's chances of passing the screening decrease.

Step 6: Content Quality Analysis

The system evaluates the quality of written content.

The following checks are performed:

1. Sentence Clarity
Detects incomplete or vague statements.

2. Action Verbs
Checks usage of professional verbs such as *developed, implemented, designed, analyzed.*

3. Repetition Detection
Identifies repeated words and redundant statements.

4. Length Analysis
Measures whether descriptions are too short or unnecessarily long.

Clear and concise resumes are more likely to be positively evaluated by recruiters [6].

Step 7: Resume Structure Evaluation

The structural format of the resume is analyzed.

The system checks:

- Proper section ordering
- Balanced content distribution
- Readability
- Logical formatting

A properly structured resume improves both machine readability and human readability. Structured resumes have significantly higher acceptance rates in automated screening [7].

Step 8: Scoring Mechanism

After evaluation, the system generates scores across multiple categories:

- Overall Resume Score
- ATS Compatibility Score
- Content Quality Score
- Skills Score
- Structure Score
- Tone & Style Score

Each category is calculated based on weighted parameters such as keyword density, section presence, and clarity.

The final score is computed using a weighted formula:

Final Score = (Skills + Content + Structure + ATS + Tone) / 5

The scoring system allows users to measure improvement when they modify their resumes.

Step 9: Feedback and Suggestion Generation

Instead of rejecting the resume, the system produces **actionable suggestions**.

Examples of suggestions:

- Add a projects section
- Include measurable achievements
- Use stronger action verbs
- Add relevant technical skills
- Improve formatting

Providing interpretable feedback increases user understanding and helps them improve application success rates [8].

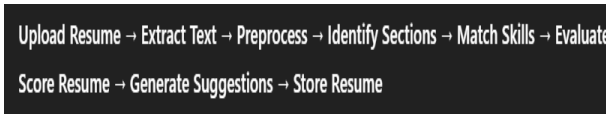
Step 10: Cloud Storage (Resume Management)

The system includes a cloud storage module where users can save multiple resumes.

Purpose:

- Store resumes for different companies
- Maintain versions
- Retrieve and compare scores

This enables candidates to create tailored resumes for various job roles instead of using a single generic resume.



6. System Architecture Explanation

The proposed Personalized AI Resume Analyzer follows a **modular web-based architecture**. The system is designed as an intelligent client-side application supported by analysis modules and optional cloud storage. Instead of functioning like a recruiter's ATS server, the architecture is designed around the **candidate as the primary user**, where processing, evaluation, and feedback are centered on helping the applicant improve their resume.

The architecture is divided into three major layers:

1. User Interface Layer (Frontend)
2. Processing and Analysis Layer (AI Engine)
3. Storage and Data Management Layer (Cloud Backend)

---

1. User Interface Layer (Frontend)

The frontend is developed using **JavaScript and JSX (React framework)**.
Although the same system could have been easily implemented in Python using backend frameworks, the project intentionally uses modern web technologies to create a real-time interactive browser application and make the project technically challenging.

The frontend performs the following functions:

- Accepts resume upload (PDF)
- Displays analysis progress
- Shows resume scores
- Displays improvement suggestions
- Allows users to manage multiple resumes

The browser directly interacts with the analysis modules and presents results instantly. React is chosen because component-based architecture improves maintainability and reusability of UI elements [9].

Key components:

- Upload Page
- Resume Analyzer Page
- Feedback Display Panel
- Dashboard (Saved Resumes)

Unlike traditional systems where processing happens entirely on a server, a significant portion of the logic is handled in the web application itself, making the tool lightweight and accessible.

---

2. Processing and Analysis Layer (AI Engine)

This layer acts as the **intelligence core** of the system.

After a resume is uploaded, the file is processed through a sequence of modules:

1. PDF Parsing Module
2. Text Processing Module
3. NLP Analysis Module
4. Scoring Engine
5. Suggestion Generator

(a) PDF Parsing Module

The parsing module extracts readable text from the uploaded PDF file.
The module converts document objects into raw text that can be interpreted by the analyzer. Document parsing is a crucial stage in automated resume processing systems [2].

(b) Text Processing Module

The extracted text is cleaned and normalized.
The module performs tokenization and filtering to remove noise and prepare the data for evaluation.

(c) NLP Analysis Module

The Natural Language Processing module analyzes resume content.
It evaluates:

- Skill keywords
- Section presence
- Writing quality
- Keyword relevance

NLP allows machines to understand textual documents and is widely used in resume screening systems [3].

(d) Scoring Engine

The scoring engine evaluates the resume based on predefined rules.

It calculates:

- ATS compatibility
- Structure quality
- Skill relevance
- Writing clarity

Rule-based scoring combined with linguistic analysis improves resume evaluation accuracy [5].

(e) Suggestion Generator

The suggestion generator converts system findings into human-readable feedback.
Instead of only giving a numeric result, the system explains **why the score is low and how to improve it**, which is the primary difference between this system and recruiter ATS systems.

---

3. Storage and Data Management Layer (Cloud Backend)

The system includes an optional cloud storage module.

Purpose of storage:

- Save multiple resumes
- Maintain versions for different companies
- Track score improvements

Each resume is stored with:

- File path
- Resume score
- Feedback data

Cloud-based document storage allows users to access resumes from any device and manage career documents efficiently [10].

Overall Architecture Workflow

The working flow of the system architecture is:

```
User uploads resume → Frontend receives file → Parsing module extracts text → NLP module analyzes → Scoring engine evaluates → Suggestions generated → Results displayed → Resume optionally saved in cloud
```

This architecture ensures:

- Fast feedback
- User transparency
- Personalized improvement guidance

Key Architectural Advantage

Typical Applicant Tracking Systems are **filtering systems** — they eliminate candidates.

This system is a **guidance system** — it educates candidates.

Instead of deciding *"you are rejected"*, the system tells the user:

"Here is why your resume may be rejected, and here is how to fix it."

This makes the architecture candidate-centric rather than recruiter-centric, bridging the gap between applicants and automated hiring technology.

7. Implementation Details

The Personalized AI Resume Analyzer is implemented as a **web-based application** using modern client-side technologies. The project was intentionally developed using **JavaScript and JSX (React)** instead of Python-based AI frameworks to create a challenging implementation aligned with the programming languages currently studied in the curriculum. While Python provides simpler libraries for text processing and machine learning, the project demonstrates that similar intelligent behavior can be achieved within a browser-based environment using web technologies.

The implementation is divided into multiple modules corresponding to the system architecture.

---

1. Development Environment

The application was developed using the following tools:

- Visual Studio Code / WebStorm IDE
- Node.js runtime environment
- Vite build tool for React applications

- Modern web browsers (Chrome/Edge) for testing

Node.js provides a package ecosystem for managing libraries and dependencies required by modern web applications [9].

---

2. Frontend Implementation (React + JSX)

The user interface is built using the **React framework** with JSX syntax. React allows the interface to be divided into reusable components, which improves code organization and scalability.

Major frontend components:

- **Home Page (Dashboard):** Displays previously analyzed resumes
- **Upload Page:** Allows user to upload resume PDF
- **Resume Card Component:** Shows resume score summary
- **Feedback Section:** Displays detailed suggestions

React's component-based design helps manage complex UI systems effectively and supports interactive applications [9].

Key frontend features implemented:

- File upload using HTML file input
- Dynamic rendering of scores
- Navigation between pages
- User feedback display

---

3. PDF Parsing Module

The system accepts resumes in **PDF format**.
To process the document, the project uses a JavaScript document parsing library.

Working process:

1. The uploaded PDF file is read as a binary object.
2. The file is converted into an ArrayBuffer.
3. Each page is processed individually.
4. Text content is extracted and combined.

Document parsing converts a structured document into machine-readable text for analysis [2].

This allows the system to interpret resumes without manual copying of content.

---

4. Resume Analysis Engine

After text extraction, the resume content is passed to the **analysis module**.
This module performs rule-based Natural Language Processing.

The system evaluates resumes based on the presence and quality of important resume sections:

- Education
- Skills
- Projects
- Experience
- Contact details

It also checks:

- Keyword presence
- Content completeness
- Writing clarity

NLP-based document analysis is widely used for automated understanding of textual data [3].

---

5. Scoring Mechanism

The system assigns a score to the resume using a weighted evaluation method.

Each category contributes to the final score:

Parameter	Evaluation Purpose
ATS Compatibility	Checks keyword relevance
Content Quality	Checks completeness of information
Structure	Checks formatting and organization
Skills	Checks relevance of technical skills
Tone & Style	Checks professional writing

The total score is calculated and presented as a percentage.

Rule-based scoring systems are commonly used in automated evaluation applications where interpretability is important [5].

---

6. Feedback and Suggestion Generation

The analyzer generates human-readable suggestions such as:

- Add missing sections
- Improve skill keywords
- Provide project descriptions
- Improve formatting
- Improve professional tone

Unlike traditional ATS systems that silently reject resumes, this system provides **actionable guidance** to the candidate.

This is the core objective of the project — improving employability rather than filtering applicants.

---

7. Cloud Storage Integration (Optional Feature)

The project also includes a cloud storage module where users can store multiple resumes for different companies.

Stored data includes:

- Resume file
- Resume score
- Feedback information

This allows a candidate to maintain multiple resume versions tailored to different job roles. Cloud document storage improves accessibility and data persistence [10].

---

8. Error Handling and Validation

The system includes several validation mechanisms:

- Accepts only PDF files
- Checks file readability
- Detects empty or image-only resumes
- Handles analysis failure

Proper input validation is important in web applications to prevent incorrect processing and system crashes.

7. Implementation Details

The Personalized AI Resume Analyzer is implemented as a **web-based application** using modern client-side technologies. The project was intentionally developed using **JavaScript and JSX (React)** instead of Python-based AI frameworks to create a challenging implementation aligned with the programming languages currently studied in the curriculum. While Python provides simpler libraries for text processing and machine learning, the project demonstrates that similar intelligent behavior can be achieved within a browser-based environment using web technologies.

The implementation is divided into multiple modules corresponding to the system architecture.

---

1. Development Environment

The application was developed using the following tools:

- Visual Studio Code / WebStorm IDE
- Node.js runtime environment
- Vite build tool for React applications
- Modern web browsers (Chrome/Edge) for testing

Node.js provides a package ecosystem for managing libraries and dependencies required by modern web applications [9].

---

2. Frontend Implementation (React + JSX)

The user interface is built using the **React framework** with JSX syntax. React allows the interface to be divided into reusable components, which improves code organization and scalability.

Major frontend components:

- **Home Page (Dashboard):** Displays previously analyzed resumes
- **Upload Page:** Allows user to upload resume PDF
- **Resume Card Component:** Shows resume score summary
- **Feedback Section:** Displays detailed suggestions

React's component-based design helps manage complex UI systems effectively and supports interactive applications [9].

Key frontend features implemented:

- File upload using HTML file input
- Dynamic rendering of scores
- Navigation between pages
- User feedback display

---

3. PDF Parsing Module

The system accepts resumes in **PDF format**.
To process the document, the project uses a JavaScript document parsing library.

Working process:

1. The uploaded PDF file is read as a binary object.
2. The file is converted into an ArrayBuffer.
3. Each page is processed individually.
4. Text content is extracted and combined.

Document parsing converts a structured document into machine-readable text for analysis [2].

This allows the system to interpret resumes without manual copying of content.

---

4. Resume Analysis Engine

After text extraction, the resume content is passed to the **analysis module**.
This module performs rule-based Natural Language Processing.

The system evaluates resumes based on the presence and quality of important resume sections:

- Education
- Skills
- Projects
- Experience
- Contact details

It also checks:

- Keyword presence
- Content completeness
- Writing clarity

NLP-based document analysis is widely used for automated understanding of textual data [3].

---

5. Scoring Mechanism

The system assigns a score to the resume using a weighted evaluation method.

Each category contributes to the final score:

Parameter	**Evaluation Purpose**
ATS Compatibility	Checks keyword relevance
Content Quality	Checks completeness of information
Structure	Checks formatting and organization
Skills	Checks relevance of technical skills
Tone & Style	Checks professional writing

The total score is calculated and presented as a percentage.

Rule-based scoring systems are commonly used in automated evaluation applications where interpretability is important [5].

---

6. Feedback and Suggestion Generation

The analyzer generates human-readable suggestions such as:

- Add missing sections
- Improve skill keywords
- Provide project descriptions
- Improve formatting
- Improve professional tone

Unlike traditional ATS systems that silently reject resumes, this system provides **actionable guidance** to the candidate.

This is the core objective of the project — improving employability rather than filtering applicants.

---

7. Cloud Storage Integration (Optional Feature)

The project also includes a cloud storage module where users can store multiple resumes for different companies.

Stored data includes:

- Resume file
- Resume score
- Feedback information

This allows a candidate to maintain multiple resume versions tailored to different job roles. Cloud document storage improves accessibility and data persistence [10].

8. Error Handling and Validation

The system includes several validation mechanisms:

- Accepts only PDF files
- Checks file readability
- Detects empty or image-only resumes
- Handles analysis failure

Proper input validation is important in web applications to prevent incorrect processing and system crashes.

9. Novelty

The proposed system introduces a user-centric perspective to resume analysis, which differentiates it from most existing resume screening tools. Conventional resume analyzers and Applicant Tracking System (ATS) simulators primarily operate from a recruiter's point of view; they simply filter or score resumes based on keyword matching and formatting compliance. In contrast, this project is designed specifically for job applicants. Instead of only generating a numerical rating, the system analyzes weaker sections of the resume and produces constructive suggestions to improve them. The feedback is personalized and actionable, guiding the user on how to enhance skills presentation, structure, wording, and keyword usage. This transforms the tool from a passive evaluation system into an interactive learning assistant that helps candidates iteratively refine their resumes before submission.

Another novel aspect of the project is the inclusion of a cloud-based resume management approach. The application is intended not only to evaluate a single resume but also to allow users to maintain multiple resumes tailored to different companies or job roles. By integrating cloud storage and web APIs, users can upload, store, and review several versions of resumes simultaneously. This supports real-world job application practices, where candidates customize resumes according to job descriptions and company expectations.

Additionally, the project intentionally adopts a JavaScript and JSX-based implementation rather than the more common Python-based machine learning development approach. Although Python provides many ready-made libraries for Natural Language Processing, the system is developed using web technologies to align with the academic curriculum and to explore the flexibility of browser-side processing and modern web frameworks. This decision increases implementation complexity but demonstrates the feasibility of performing document parsing, text analysis, and AI-assisted feedback generation directly within a web application environment. The combination of candidate-oriented feedback, cloud-supported resume management, and a full JavaScript implementation establishes the originality and practical contribution of the proposed system.

10. Conclusion

The Personalized Resume Analyzer project presents an intelligent web-based solution designed to assist job applicants in improving the quality and effectiveness of their resumes before applying for jobs. Unlike traditional recruitment software that filters or rejects candidates, the proposed system acts as a candidate-support tool, helping users understand how their resumes are interpreted by automated screening mechanisms such as Applicant Tracking Systems (ATS).

The system accepts resumes in PDF format, extracts textual information, evaluates the content, and provides structured feedback along with a numerical performance score. The analysis considers multiple parameters including resume structure, clarity, skills representation, writing style, and keyword relevance. Based on these parameters, the application generates improvement suggestions that guide candidates in modifying their resumes according to professional standards.

The project demonstrates how Artificial Intelligence and Natural Language Processing can be applied not only for automation but also for education and career preparation. Many job applicants are rejected during early screening stages because they are unaware of formatting requirements or keyword expectations. By making the evaluation process transparent, the system helps bridge the gap between recruitment technology and applicants.

A unique aspect of this project is the choice of technology stack. While similar systems are commonly implemented in Python due to its strong machine learning ecosystem, this project intentionally uses JavaScript and JSX for both frontend interaction and AI integration. The objective was to create a challenging implementation that demonstrates AI capability within modern web development frameworks. The inclusion of optional cloud storage for managing multiple resumes tailored to different companies further extends the practical usefulness of the application.

The system does not replace recruiters or hiring decisions. Instead, it prepares candidates by improving resume quality, thereby increasing the likelihood of passing automated screening stages and reaching human evaluation. The analyzer functions as a learning tool that teaches proper resume writing practices rather than simply judging applicants.

Overall, the project highlights a real-world application of AI in employability enhancement. It shows that intelligent systems can be designed to empower users instead of eliminating them from processes. By providing feedback, transparency, and guidance, the Personalized Resume Analyzer supports students and job seekers in presenting their qualifications effectively and confidently.

11. References

[1] J. D. Kelleher and B. Tierney, *Data Science*. Cambridge, MA, USA: MIT Press, 2018.

[2] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. Sebastopol, CA, USA: O'Reilly Media, 2009.

[3] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. Draft, Stanford University, 2023.

[4] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. New York, NY, USA: McGraw-Hill, 2019.

[5] M. Russell and M. Klassen, *Mining the Social Web: Data Mining Facebook, Twitter, LinkedIn, Instagram, GitHub, and More*, 3rd ed. Sebastopol, CA, USA: O'Reilly Media, 2019.

[6] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2013, pp. 3111–3119.

[7] J. Brownlee, *Deep Learning for Natural Language Processing*. Machine Learning Mastery, 2017.

[8] Mozilla Foundation, "PDF.js Documentation," Available: https://mozilla.github.io/pdf.js/ (accessed Jan. 2026).

[9] React Team, "React — A JavaScript library for building user interfaces," Available: https://react.dev/ (accessed Jan. 2026).

[10] T. Bray, “The JavaScript Object Notation (JSON) Data Interchange Format,” RFC 8259, Internet Engineering Task Force (IETF), 2017.

[11] M. L. P. (LinkedIn Engineering), “Applicant Tracking Systems and Resume Parsing Techniques,” LinkedIn Talent Solutions Technical Report, 2022.

[12] A. Valentine, D. Raychaudhuri, and I. Seskar, “Machine learning-assisted adaptive systems for modern network environments,” *arXiv preprint*, 2025.